

disconnect any number of network drives you mapped, thus not requiring you to manually do it every time you shut down your computer. Remember, simply because this is the opposite of the last script, it won't need your drives to have been previously created by WSH. Here's the script.

```
Set WshNetwork =
WScript.CreateObject("WScript.Network")
WshNetwork.RemoveNetworkDrive
"S:", True
WshNetwork.RemoveNetworkDrive
"T:", True
```

The first line is used to declare that we want to make changes to the network subsystem. This is followed by the function used in the network object, which is to remove a network drive. This function is followed by the network drive you want to disconnect. Enter this command however many times you want to disconnect more drives; just remember to replace the drive letter. Disconnect commands are executed in order from top to bottom, as usual.

The "True" Boolean at the end of the script indicates to WSH to disconnect the drive even if it's still in use. You can change this Boolean to "False", or not enter it at all, to prevent WSH from disconnecting drives in use.

Create or edit Registry values

This script will allow you to create one or more Registry values using a single file. This can be useful when dealing with tweaks concerning the registry, which require you to create values. The script:

```
Set WshShell =
WScript.CreateObject("WScript.Shell")
WshShell.RegWrite
"HKLM\Software\new\text", "Normal
text", "REG_SZ"
WshShell.RegWrite
"HKLM\Software\new\Numbered
value", 543, "REG_DWORD"
```

As always, the first line declares the object. The function in the next line is RegWrite, which indicates we want to make changes to the Registry. Next comes the path to the values you want to create, followed by the data you want to input. Finally comes the value type: REG_SZ indicates that it be a string value, REG_DWORD indicates a DWORD number (the value will be stored in decimal format), REG_BINARY indicates a binary value, and so on.

Also, don't be confused by the "HKLM"; this expands to HKEY_LOCAL_MACHINE. For

HKEY_CURRENT_USER, you can use HKCU, and so on. Remember that the steps to create and edit values are the same. This script can create a single file full of all your custom tweaks for the Registry with a mere double-click.

Delete Registry keys

Using this script, you can delete multiple registry entries using a single file. This should allow you to delete even entire keys. Here's the script for it. Enter it in Notepad and save it with the .vbs extension.

```
Set WSHShell=WScript.CreateObject
("WScript.Shell")
WSHShell.RegDelete
"HKCU\Software\EAG\"
WSHShell.RegDelete
"HKCU\Software\EAG\username"
```

The first line declares it's going to make changes to the shell. The second line—WSHShell.RegDelete—is self-explanatory and asks WSH to delete a Registry key. The key to be deleted is entered after this command in double quotes. You can add as many entries as you want.

We have added two commands for deleting; this is for the sake of explaining the difference between deleting values (the entries on the right-hand pane in the registry) and keys (the folder-like entries on the left-hand pane). The only difference between the two is that you have to include a "\" at the end of the path when deleting a key. For example, "HKCU\software\value" will remove a value called "value", while "HKCU\software\value\" will delete a key called "value".

Open important documents located in network shares

As you probably are now aware of the general format of the scripts, this one is an example of how you can combine two scripts to make tasks easier. This script will first map a network share to a drive and then open a document located on it.

```
Set WSHNetwork =
WScript.CreateObject("WScript.Network")
WSHNetwork.MapNetworkDrive
"S:", "\\network\share 1"
WSHShell.Run "S:\reports.doc"
WSHShell.Run "\\network\share 1"
```

The first line is what we explained earlier, declaring the network object and then including the "MapNetworkDrive" function. The second line should map your network share to a drive letter, and the other lines, containing WSHShell.Run, will open a document located in your network shares. Note that one of the lines

containing the Run function has S:, a drive letter that was mapped earlier in the script. Were you to put this line on the top, WSH wouldn't be able to determine where S: is located. This example is solely to explain that you should indeed plan out your scripts as they occur in order from top to bottom. (Yes, we said that already, but it needed mentioning once more.)

Start the Registry editor without remembering the last used key

How often do you hate it upon starting the Registry editor, that the path you were last working on shows up directly in the window? With this script, you can start regedit without opening the last window used. It works by first removing an entry in the Registry that contains the last-used key path, thus clearing it, and then starting regedit.

```
Set WSHShell=WScript.CreateObject
("WScript.Shell")
```

```
WSHShell.RegDelete
"HKCU\Software\Microsoft\Windows\CurrentVersion\Applets\Regedit\LastKey"
```

```
WSHShell.Run "REGEDIT"
```

The first line declares the "WSHShell" object; this is necessary whenever you're working with scripts that make changes to the Registry or to individual files.

The second line contains the major change, which is to delete the value storing the path to the last-used key; it naturally uses the "regdelete" function, which was explained earlier.

Finally, once the last-used path in the Registry is cleared out, the simple "WSHShell.Run" function explained earlier will start regedit at its root folder.

Mix it up

That's pretty much enough to get the feel of scripting; you can also combine several scripts together to form a single one, like we've seen in the last two examples. Remember that scripts run in order from top to bottom: So make sure you plan out your scripts properly since the earlier functions won't be aware of the commands below them and won't be able to execute if they depend on the functions at the end.

If you wish to go deeper into scripting, check out msdn.microsoft.com and search for Windows Scripting Host to get a rather lengthy manual containing all the functions possible. It won't be long before you become so addicted that you will end up writing lengthy scripts for simple tasks instead of executing them normally—just for the sake of it! ■

mayur_bhatia@thinkdigit.com